What Do the Rules Prove? Verifying Claims in *The Resistance: Avalon* with a Constraint Solver

[Author name]

Programming course project (Fall 2026)
[Student ID, school]

Abstract. In a game of hidden roles most claims sound plausible, but few are actually forced by the rules. We compile the rulebook of *The Resistance: Avalon* into 31 clause-tagged constraints and use the Z3 SMT solver to decide, for a game record, a point of view and a claim, whether the claim is *entailed*, *contradicted* or *undetermined*, or whether the record itself breaks the rules. Every verdict comes with evidence a person can check, either a minimal set of rule clauses or two complete role assignments that disagree. For an undetermined claim, a counterexample-guided hitting-set search also finds the fewest players whose alignment would settle it. We test the formal model three ways: against an independent brute-force implementation (1,296 of 1,296 random verdicts agree), on 44 hand-written natural-language cases, and by auditing 12,447 real online games, which exposed one illegal record. We then ask where a language model fits. On 510 questions built from real games, Qwen3-8B is right 50.0% of the time when it answers directly and 67.1% after thinking with a 16k-token budget, but 92.0% when it only translates the question into the formal model and Z3 decides. Letting the model review the solver's verdict afterwards lowers accuracy to 89.2%, because it accepts the solver's evidence even when its own translation was wrong. The lesson is a division of labour: the model translates, the solver reasons, and people should check the solver's evidence rather than the model's confidence.

Keywords: Constraint solving · SMT · Verification of conclusions · Explanations · Hidden-role games · LLM autoformalisation

1 Introduction

A five-player game of Avalon has just played its first quest. Two players went on it, and when the cards were shuffled and turned over, one of them read *Fail*. Is seat 1 evil? A player who answers "probably" is reasoning about people. A verifier has to answer about rules, and the rules say only that *at least one* of the two quest members is evil. They do not say which. This report is about measuring that gap, between what sounds right and what the rules force, with a constraint solver.

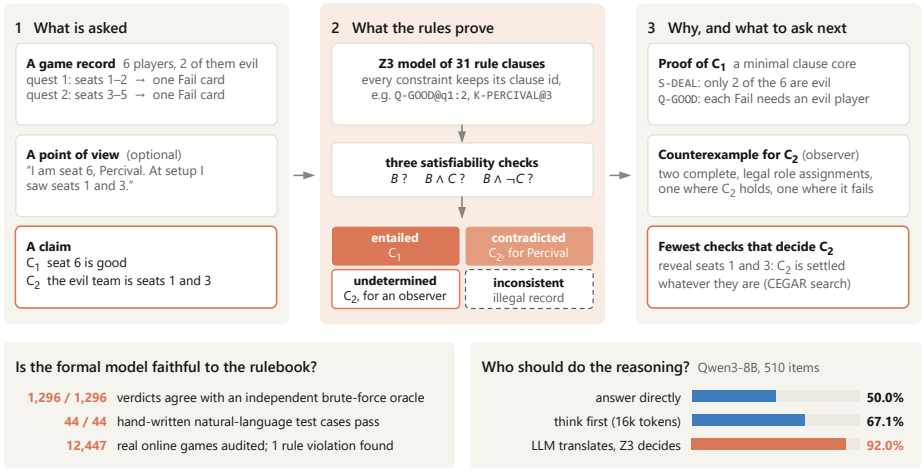


Fig. 1. What this project does. (1) A question is a game record, an optional point of view and a claim. (2) The rulebook is compiled once into a Z3 model whose constraints keep their clause identifiers, and three satisfiability checks give one of four verdicts. The same claim C_2 is undetermined for a public observer but contradicted for Percival, who saw seats 1 and 3 at setup and knows that one of them is Merlin. (3) Each verdict comes with evidence a person can check, and an undetermined claim also comes with the fewest alignment checks that would decide it. Bottom: how we tested the model (Section 5) and the main result of the language-model study (Section 7).

We chose *The Resistance: Avalon* [7] instead of a regulation because its rulebook has everything the project asks for: tables indexed by player count, numeric thresholds, an exception (the fourth quest of a large game needs two Fail cards), and rules that only bite in combination. It adds two things that make verification harder and more interesting. First, information is *private*, so the same claim can be settled for one player and open for another. Second, there is a public log of 12,447 real games [2] to test the formal model against.

We set out to answer four questions.

Q1 Can a rulebook be turned into a checker that says not only *whether* a conclusion follows, but *why*, and *what* would settle it when it does not? (Sections 3 and 4)

Q2 How do we know the formal model says what the rulebook says? (Section 5)

Q3 What do the rules alone let real players deduce? (Section 6)

Q4 Where in such a pipeline does a language model help? (Section 7)

Figure 1 shows what we built and the two headline results.

Key findings

– **Four verdicts, not three.** A record can break the rules. Without a separate check such a record makes every claim vacuously "entailed". Our audit of

real logs found one illegal game, and in the language-model study the same check turned all 27 of the model's role-configuration errors (of its 55 imperfect translations) into visible warnings.
- **Every verdict is checkable.** Proofs are subset-minimal sets of clause instances, and open claims come with two complete role assignments. In real games, two to four alignment checks settle who is evil, and a counterexample-guided search finds the smallest such set in under a second.
- **The rules settle little for the good side.** After five quests a Loyal Servant can pin down on average 10% of the other players' alignments, while a Minion of Mordred knows 98%. If only quest results were announced, and not the number of Fail cards, the Servant's figure would drop to 2%.
- **Let the model translate, not reason.** The same 8B model scores 50.0% when it answers directly and 67.1% when it thinks first, but 92.0% when Z3 does the reasoning.
- **The model defers to the solver even when the solver is wrong.** Shown the solver's evidence, it adopts the solver's verdict on 259 of the 276 items it is asked again, including 37 of the 39 where the solver had been given a mistranslated question.

2 The Material and What We Formalise

The rulebook. The material is the official rulebook of *The Resistance: Avalon* [7]. We checked its tables against two public rule summaries. The rules that matter for verification fall into six groups: *setup* (how many evil players there are for each player count, one role card per player, optional special roles); *private sight* (who sees whom when the game starts); *team building* (team sizes per quest, majority voting, a leader who rotates after every proposal, and an evil win after five rejected proposals in a row); *quests* (good players must play Success; the number of Fail cards is revealed; one Fail fails a quest, except that the fourth quest in games of seven or more players needs two); the *end of the game* (three failed quests, or three successful ones followed by the assassin's attempt to find Merlin); and the optional *Lady of the Lake*, which lets its holder learn one player's alignment privately.

What is formalised. Table 1 lists all 31 clauses with their formal reading and the hand-written tests that exercise them. The full version, with the rulebook section of each clause, is `RULES.md` in the code. Each clause is instantiated once per position it applies to: per seat, per quest, per proposal or per use of the Lady. Every instance keeps a tag such as `Q-GOOD@q3:5` ("in quest 3, seat 6 plays Success if good"), and these tags are what explanations are made of (Section 3.3).

What is not formalised, and why. Speech, accusations and bluffs are left out, because the rules do not constrain what anyone says. Votes are left out as evidence about roles, because any vote is legal for any role. We check this property directly: deleting all votes from a record never changes a single verdict (246 of

Table 1. The 31 formalised clauses. n: number of players; $cfg(r)$: copies of role r in play; *seen*: the seats a player saw at setup; T_q, f_q: team and number of Fail cards of quest q. Tests refer to Table 5; "random" and "corrupt" to the checks of Section 5.

Clause	Rule (paraphrase)	Formal reading	Tests
Setup			
`S-SIZE`	5 to 10 players	$n \in \{5, \dots, 10\}$	random
`S-COUNT`	evil players: 2, 2, 3, 3, 3, 4 for $n = 5, \dots, 10$	$\sum_{r \in Evil} cfg(r) = E(n)$	T01–T03
`S-TOTAL`	one role card per player	$\sum_r cfg(r) = n$	corrupt
`S-UNIQUE`	each special role at most once	$cfg(r) \leq 1$ for special r	—
`S-DEAL[`r`]`	the role cards are dealt to the seats	$\sum_i [role_i = r] = cfg(r)$	T01
Private sight			
`V-SELF`	each player knows their own role	$role_p = \rho$	T33, T35
`K-EVIL`	evil players other than Oberon see each other	$(evil_j \wedge role_j \neq \textsc{Oberon}) \leftrightarrow j \in seen$	T31, T32
`K-MERLIN`	Merlin sees the evil players except Mordred	$(evil_j \wedge role_j \neq \textsc{Mordred}) \leftrightarrow j \in seen$	T26–T28, T43
`K-PERCIVAL`	Percival sees Merlin and Morgana, without telling them apart	$role_j \in \{\textsc{Merlin}, \textsc{Morgana}\} \leftrightarrow j \in seen$	T29, T30, T34
`K-NONE`	Loyal Servants and Oberon see nobody	$seen = \emptyset$	random
`K-SELF`	nobody is listed as seeing themself	$p \notin seen$	random
`K-LADY`	the Lady's holder learns the target's alignment	$evil_t$ = reported result	T40
Team building			
`T-LEADER`	leadership passes clockwise after each proposal	$\ell_{k+1} = \ell_k + 1 \bmod n$	T16
`T-SIZE`	team size by player count and quest	$\lvert team \rvert = SIZE[n][q]$	T04, T05
`T-DISTINCT`	a player holds at most one team token	team members are distinct	random
`T-VOTE`	a proposal passes on a strict majority; a tie rejects	$approved \leftrightarrow 2 \cdot \#yes > n$	T12–T14
`T-HAMMER`	five rejections in one round and evil wins	≤ 5 proposals; the 5th rejection ends the game	T15
`T-LAST`	the approved proposal is the quest team	last proposal approved and equal to T_q	corrupt
Quests			
`Q-SIZE`	the quest team has the table size	$\lvert T_q \rvert = SIZE[n][q]$	T06
`Q-GOOD`	good players must play Success	$\neg evil_i \rightarrow \neg fail_{q,i}$ for $i \in T_q$	T17–T25
`Q-COUNT`	the number of Fail cards is revealed	$\sum_{i \in T_q} fail_{q,i} = f_q$	T17–T25
`Q-RESULT`	one Fail fails a quest; quest 4 with $n \geq 7$ needs two	$failed_q \leftrightarrow f_q \geq need(n, q)$	T07–T11
`R-ORDER`	quests are played in order (no Targeting)	the k-th quest record has index k	corrupt
End of the game			
`G-END`	the game stops at three successes or three failures	no quest after the third win or loss	T39
`G-OUTCOME`	three failed quests and evil wins	3 failures $\rightarrow$ evil wins	random
`G-ASSASSIN`	the assassination happens only after three successes	assassination $\rightarrow$ wins ≥ 3	T38
`G-MERLIN-HIT`	evil wins if and only if the assassin picks Merlin	$role_a = \textsc{Merlin} \leftrightarrow$ evil wins	T36, T37
Lady of the Lake			
`L-WHEN`	used after quests 2, 3 and 4	quest index $\in \{2, 3, 4\}$, increasing, already played	—
`L-FIRST`	starts with the player to the right of the first leader	$holder_1 = \ell_1 - 1 \bmod n$	T41
`L-CHAIN`	the player checked receives the Lady	$holder_k = target_{k-1}$	T41
`L-PREV`	a former holder cannot be checked	$target_k \notin \{holder_1, \dots, holder_k\}$	T41

246 claims, Section 5). Whether an evil player chooses to play Success is left free, because the rules allow it. The optional *Targeting* variant and behavioural rules such as "do not show your card" are not modelled either. This is a deliberate choice with a consequence: our verdicts are about what the rules *force*, not what is *likely*. Section 6 shows that human assassins beat a rules-only guess, which tells us they use exactly the information we chose not to model.

Readings the rulebook leaves open. Formalising a rulebook exposes sentences that can be read more than one way. We found four (Table 2). Rather than pick one silently, the encoding makes each a switch, and the tests pin down how verdicts change. The most consequential is `V-FAILCOUNT`. If the number of Fail cards is public, two Fail cards on a two-player team prove that both players are evil (T17). If only the quest's result is public, the same record leaves it open (T42).

Table 2. Readings the rulebook leaves open. Each is a switch in the encoding; the default is the rulebook's literal reading.

Switch	Question	Default	Effect
`V-FAILCOUNT`	Is the number of Fail cards public, or only the result?	public: the cards are shuffled and turned over	"two Fails on a team of two ⇒ both evil" turns from entailed (T17) to undetermined (T42)
`V-OBERON`	Does Merlin see Oberon?	yes: the rulebook hides only Mordred from Merlin	Merlin seeing three evil players in a 10-player game is consistent (T43) or illegal (T44)
`L-FIRST`	Who is "to the right" of the first leader?	the previous seat, since leadership moves clockwise	only affects the legality of Lady of the Lake records
`T-LEADER`	Does leadership keep rotating across quests?	yes, independently of the quest	confirmed on 75,855 of 75,855 leader changes in the real logs

3 From Rules to a Verifier

3.1 Instances, unknowns and constraints

An *instance* is a triple (R, V, C). The record R holds the public facts: the number of players n, the announced role configuration, every proposal (leader, team, approvers, outcome), every quest played (team, number of Fail cards, result), the end of the game, and any uses of the Lady of the Lake. The optional point of view V names a seat p, its role, the seats it saw at setup, and its private Lady results; without V we reason as a public observer. The claim C comes from a small language: $\mathit{evil}(i)$, $\mathit{good}(i)$, $\mathit{role}_i = r$, "at least / exactly k of the seats in S are evil", "i and j are on the same side", "the evil team is exactly S", and Boolean combinations of these.

The unknowns are a role role_i for every seat (an integer over the seven roles) and, for every member i of every played quest q, a Boolean $\mathit{fail}_{q,i}$ saying whether

i played a Fail card. We write $evil_i$ for

$$role_i \in \{\textsc{Morgana}, \textsc{Mordred}, \textsc{Oberon}, \textsc{Minion}\}.$$

Everything else in R is data. Clauses that involve unknowns become constraints. Four examples, for every role r, every quest q with member i, and every other seat j when V is Merlin's view:

$$\begin{aligned}
\texttt{S-DEAL[}r\texttt{]}: &\quad \textstyle\sum_i [role_i = r] = cfg(r),\\
\texttt{Q-COUNT@}q: &\quad \textstyle\sum_{i \in T_q} fail_{q,i} = f_q,\\
\texttt{Q-GOOD@}q\texttt{:}i: &\quad \neg evil_i \rightarrow \neg fail_{q,i},\\
\texttt{K-MERLIN@}j: &\quad (evil_j \wedge role_j \neq \textsc{Mordred}) \leftrightarrow j \in seen.
\end{aligned}$$

Clauses about the record alone (team sizes, vote majorities, leader rotation) have no unknowns. We still add each one as a tagged Boolean constant, so that a record that breaks a table entry is caught, and the broken entry appears in the explanation like any other clause.

Two encoding choices deserve a word. First, per-card Fail variables make `Q-GOOD` appear by name in proofs. The independent oracle of Section 5 instead checks the inequality "Fail cards $\leq$ evil players on the team", so the two implementations really are different formalisations of the same sentences. Second, several rules are boundary-sensitive: "more than half" is $2 \cdot \#yes > n$, not $\geq$, so a tie rejects, and the two-Fail exception applies when $n \geq 7$ and $q = 4$. Each side of each boundary has its own test (T07–T13).

3.2 Four verdicts

Let $B = B(R, V)$ be the conjunction of all clause instances. Three satisfiability checks classify a claim:

B	$B \wedge C$ and $B \wedge \neg C$	verdict
UNSAT	—	*inconsistent*: the record (or the view) breaks the rules
SAT	SAT and UNSAT	*entailed*: C holds in every legal role assignment
SAT	UNSAT and SAT	*contradicted*: C fails in every legal role assignment
SAT	SAT and SAT	*undetermined*: the rules leave C open

Proposition 1. *Exactly one verdict applies to every instance. Moreover, C is entailed (contradicted) if and only if it is true (false) under every assignment of roles to seats that is consistent with all clauses, the record and the point of view.*

The proof is immediate from the definitions, because B is satisfiable exactly when some legal assignment exists. The first check matters more than it looks. If B is unsatisfiable, then $B \wedge C$ and $B \wedge \neg C$ are both unsatisfiable, and a two-check verifier would call every claim both entailed and contradicted. Treating *inconsistent* as a verdict of its own turns this vacuous truth into a warning. In

our experiments that warning caught a corrupted real log (Section 5.3) and the language model's translation errors (Section 7.4).

Two further outcomes are worth naming. The solver may return *unknown*. Our encodings are small finite-domain problems, and with a 60 s limit per query this never happened; the code raises an error rather than guessing. And the verdict is a property of (R, V, C), not of (R, C): the same claim can be open for the public and settled for one player, so every verdict is reported together with its point of view.

3.3 Evidence

A verdict is only as useful as the reason it comes with. For an *entailed* claim, $B \wedge \neg C$ is unsatisfiable, and we ask Z3 for an unsatisfiable core over assumption literals, one per clause instance. We then shrink it by deletion: each instance is dropped in turn and kept out if the rest is still unsatisfiable [10,9]. The result is subset-minimal (no instance can be removed), though not necessarily the smallest such set. *Contradicted* claims get the same treatment with $B \wedge C$, and *inconsistent* records with B alone. For an *undetermined* claim the evidence is two models, $w^+ \models B \wedge C$ and $w^- \models B \wedge \neg C$, printed as complete role assignments. Both kinds of evidence can be checked by hand, and neither depends on trusting the solver. One practical note: when several cores or witnesses exist, which one Z3 returns depends on the queries the process has already run. Any of them is valid, but a reproduction must replay the same order (Section 8).

3.4 A worked example

The example in Figure 1 is computed by the code; the box below gives the solver's actual output.

Record. Six players; roles in play: Merlin, Percival, two Loyal Servants, Morgana, Minion (two evil). Quest 1: seats $\{1, 2\}$, one Fail card. Quest 2: seats $\{3, 4, 5\}$, one Fail card.

C_1**: "seat 6 is good"** is *entailed*. Core (10 instances): `S-DEAL[merlin]`, `S-DEAL[percival]`, `S-DEAL[servant]` (four good role cards, hence two evil players); `Q-GOOD@q1` for seats 1 and 2, `Q-COUNT@q1`, `Q-GOOD@q2` for seats 3, 4 and 5, `Q-COUNT@q2` (each Fail card was played by an evil team member, so one evil player sits in $\{1, 2\}$ and the other in $\{3, 4, 5\}$).

C_2**: "the evil team is exactly** $\{1, 3\}$**"** is *undetermined* for an observer. $w^+ =$ (Morgana, Servant, Minion, Servant, Merlin, Percival), $w^- =$ (Servant, Minion, Morgana, Servant, Merlin, Percival). Minimum reveal set: $\{1, 3\}$ (or $\{2, 3\}$), found in 3 rounds and 6 solver calls.

Change the point of view to seat 6 as Percival, who saw seats 1 and 3: C_2 becomes *contradicted*, because one of seats 1 and 3 is Merlin and therefore good.

Change the record so that quest 1 shows two Fail cards: the record becomes *inconsistent*, with the same ten instances as the core (three Fail cards on disjoint teams need three evil players, and there are two).

Algorithm 1 Minimum reveal set by counterexample-guided implicit hitting sets

Input: instance (R, V), undetermined claim C, candidate seats P

$\mathcal{D} \leftarrow \emptyset$
loop
  $H \leftarrow$ a minimum hitting set of $\mathcal{D}$ ▷ exact branch and bound
  if TwoWorld(H) is unsatisfiable **then return** H ▷ sufficient, and optimal
  $(w, w') \leftarrow$ the model found; $A \leftarrow H$
  for $j \in \Delta(w, w')$ **do** ▷ generalise: make the two worlds agree on more seats
    if TwoWorld($A \cup \{j\}$) has a model (u, u') **then**
      $A \leftarrow A \cup \{j\}$; $(w, w') \leftarrow (u, u')$
  $\Delta \leftarrow \{j \in P \setminus A : \mathit{evil}_j(w) \neq \mathit{evil}_j(w')\}$
  if $\Delta = \emptyset$ **then return** NONE ▷ no set of alignments can decide C
  $\mathcal{D} \leftarrow \mathcal{D} \cup \{\Delta\}$

4 What Would Settle It? Minimum Information Completion

An *undetermined* verdict invites a follow-up question: which extra fact would settle the claim? Avalon has a natural unit of information, since the Lady of the Lake reveals one player's alignment. So we ask for the fewest alignments whose values, *whatever they turn out to be*, decide the claim.

Sufficiency is one SAT query. Let P be the seats whose alignment the point of view does not already know. A set $S \subseteq P$ is *sufficient* for C if any two legal worlds that agree on the alignments in S also agree on C. This is a single query on two copies of the encoding:

$$\textsc{TwoWorld}(S): \quad B(x) \wedge B(y) \wedge \big(C(x) \neq C(y)\big) \wedge \textstyle\bigwedge_{j \in S} \mathit{evil}_j(x) = \mathit{evil}_j(y).$$

S is sufficient exactly when TwoWorld(S) is unsatisfiable.

A hitting-set view. For a pair of legal worlds (w, w') that disagree on C, let $\Delta(w, w') = \{j \in P : \mathit{evil}_j(w) \neq \mathit{evil}_j(w')\}$. Then S is sufficient if and only if it intersects $\Delta(w, w')$ for *every* such pair, since otherwise that pair agrees on S and still disagrees on C. The smallest sufficient set is therefore a minimum hitting set of an exponentially large, implicitly given family of sets. This is the setting of implicit hitting-set algorithms [5] and of counterexample-guided loops [4,19]. Algorithm 1 only ever materialises the sets it needs.

Proposition 2. *If Algorithm 1 returns a set H, then H is a minimum sufficient set. If it returns* NONE*, no sufficient set exists. It always terminates.*

Proof. Every set added to $\mathcal{D}$ is Δ of a pair of legal worlds that disagree on C (generalisation only replaces the pair by another such pair), so every sufficient set hits all of $\mathcal{D}$ and has at least $|H|$ elements. The returned H is sufficient

by the final check, hence minimum. If $\Delta = \emptyset$, two legal worlds agree on every candidate's alignment yet disagree on C, so no reveal can separate them. Each new Δ is disjoint from the current H, so it differs from every set already in $\mathcal{D}$ (each of which H hits). There are finitely many subsets of P, so the loop terminates.

The generalisation step is optional for correctness, but it produces smaller sets Δ and therefore stronger cuts. A claim can lack any sufficient set. "Seat 4 is Merlin" is an example when the alternatives are other good roles, because no alignment check tells Merlin from a Loyal Servant.

Adaptive checks. A player who checks one seat at a time can choose the next seat after seeing the last answer. The worst-case number of sequential checks is a two-player game against an adversarial (but legal) world. We solve it by memoised minimax search over sets of revealed alignments. Revealing the one-shot set in any order is one adaptive strategy, so the adaptive depth is never larger than the one-shot size. Section 6 shows that on real games it was never smaller either.

5 Is the Formal Model Faithful to the Rulebook?

A model that runs is not a model that is right. Writing "younger than 14" as $age \leq 14$ gives a solver that works perfectly and a verdict that is wrong exactly at 14. We therefore test the formalisation in three independent ways, each designed to catch a different kind of mistake (Table 3).

Table 3. Three independent checks of the formal model. All numbers are from the full local run (`selfcheck.py`, 2,236 s) and the audit of the logs.

Check	What it catches	Result
Hand-written cases	misreadings of single rules, boundaries ($<$ vs. $\leq$), wrong combinations	44 / 44 pass
by kind	simple 9, boundary 7, combination 7, point of view 6, illegal record 12, reading switch 3	
by verdict	entailed 19, contradicted 6, undetermined 6, inconsistent 13	
Second implementation	coding errors in the Z3 encoding: brute force over role assignments, tables typed again, quest rule as an inequality	1,296 / 1,296 agree
Soundness on simulated games	over-constraining: every entailed claim true and every contradicted claim false under the hidden roles	1,296 / 1,296
Corrupted records	under-constraining: one injected violation per record must give *inconsistent*	60 / 60
Minimum reveal sets	search errors: enumeration, CEGAR on the oracle, CEGAR on Z3; adaptive $\leq$ one-shot	44 / 44 agree
Votes carry no information	a modelling claim: deleting all votes never changes a verdict	246 / 246
Real logs	mismatch with how the game is actually played: 12,447 games audited	1 illegal game

5.1 Hand-written natural-language cases

We wrote 44 scenarios in Chinese, each with a question and a proposed answer, formalised the answer as a claim, and derived the expected verdict by hand from the rulebook before running anything. All 44 are in Appendix A. They include correct answers (entailed), wrong answers (contradicted), unsupported answers (undetermined) and questions about illegal records. The boundary cases are the ones most likely to catch a sloppy formalisation. A quest with one Fail card fails in a six-player game but not as the fourth quest of a seven-player game (T07, T08), while two Fail cards do fail it (T09) and so does one Fail on quest three (T10). Three approvals out of six reject a proposal; four out of seven pass it (T12, T13). The point-of-view cases check private sight: Merlin seeing two evil players in a game with Mordred must conclude that exactly one more is hidden (T27), and a Percival who claims to have seen three players is describing an illegal game (T34).

5.2 A second, independent implementation

`oracle.py` is a brute-force reading of the rules that shares no rule code with the Z3 encoding. It enumerates every assignment of the role multiset to the seats and keeps those consistent with the record. Its tables are typed again from the rulebook, its quest rule is the inequality "Fail cards $\leq$ evil players on the team" rather than per-card variables, and record legality is checked by ordinary loops. On 432 simulated games with three random claims each, the two implementations agree on all 1,296 verdicts. Every verdict is also *sound* with respect to the hidden roles the simulator dealt: no entailed claim is false and no contradicted claim is true. Sixty records with one injected violation each (an extra team member, more Fail cards than players, a flipped result, a vote without a majority) are *inconsistent* for both implementations. Brute-force enumeration and both CEGAR runs return minimum reveal sets of the same size on all 44 undetermined instances tried. In 7 of them no sufficient set exists, and both methods report so.

5.3 Auditing 12,447 real games

The `avalonlogs` dataset [2] stores one JSON file per online game. Two of its conventions are undocumented, so we decided them from the data rather than assuming them. The `votes` field lists the *approvers*: that reading agrees with the recorded outcome of all 88,302 proposals, while the opposite reading contradicts 81,662 of them. Leadership moves forward through the player array in all 75,855 leader changes, so the array order is the seating order. We then checked every game against every record-level clause and every revealed set of true roles against the full model. One game breaks `Q-RESULT`, and two games carry a platform field (`failsRequired`) that disagrees with the rulebook's table. More importantly, the roles revealed at the end of each game are consistent with its record in *every* game. A model that over-constrained, for example one that let Merlin see Mordred, would have rejected some real games.

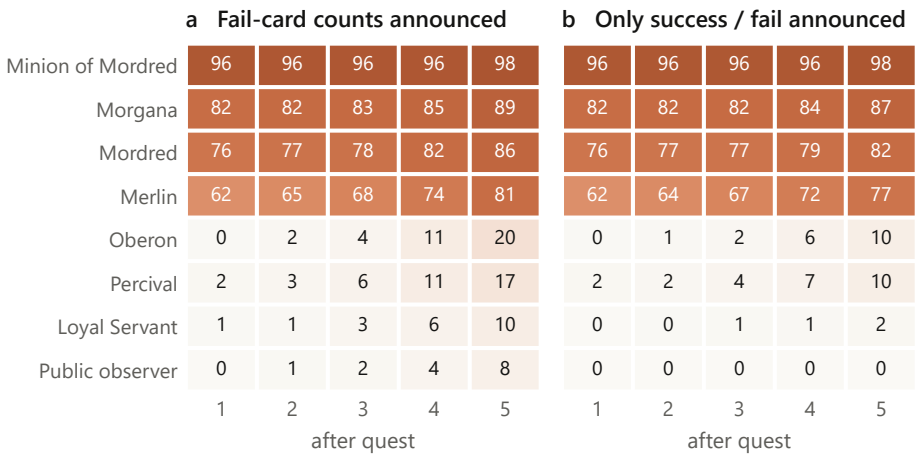


Fig. 2. The rules settle almost nothing for the good side, and hiding the Fail count erases most of what they do settle. Share (%) of the other players whose alignment the rules fix, by point of view and quest, averaged over 600 real games. (a) The rulebook's reading, in which the number of Fail cards is announced. (b) The alternative reading in which only success or failure is announced.

6 What Do the Rules Prove in Real Games?

With a verifier we trust, we can ask how much the rules themselves reveal during real play. We sampled 600 legal games (seed 271). After each quest, and for each player's true point of view as well as the public observer's, we counted how many of the other players' alignments are *entailed*, that is, fixed by the rules.

Knowledge is lopsided from the start (Figure 2). Evil players know their team from setup: a Minion of Mordred has 96% of the board fixed before any quest, rising to 98%. Merlin starts at 62% (Mordred is hidden from him) and reaches 81%. The good players who see nobody gain information slowly: a Loyal Servant goes from 1% to 10% over five quests, and a public observer from 0% to 8%. Oberon, who is evil but sees nobody, does only a little better (0% to 20%). The reading of the rulebook matters. If only quest results were announced, a Servant would end the game with 2% of the board fixed instead of 10%, and the observer with essentially nothing. The Fail count is most of what the rules give the good side.

The assassin always has to guess (Figure 3a). In 387 games that ended in an assassination, we looked at the board from the assassin's true point of view just before the shot. The rules leave on average 4.2 players who could be Merlin, and never exactly one. Human assassins still hit Merlin 38.5% of the time, against 25.3% for a uniform pick among the logically possible Merlins, so they extract signal from behaviour, the part of the game we chose not to model. In two games the assassin shot a player whom the rules had already ruled out as Merlin.

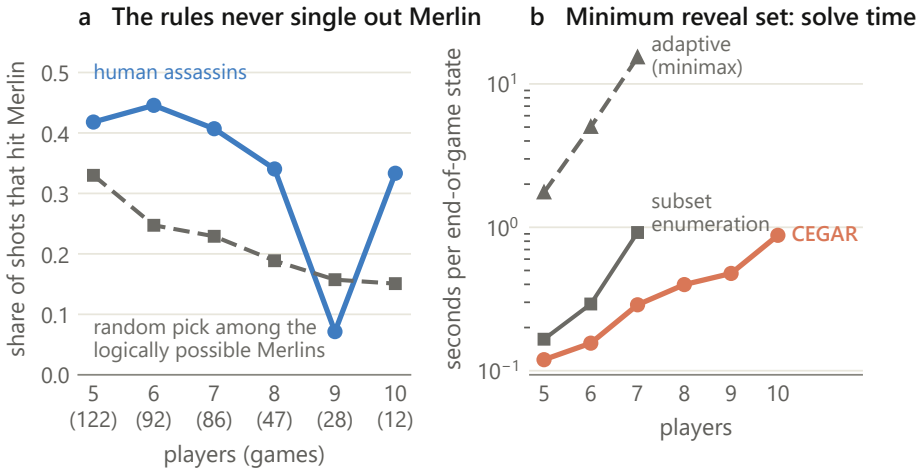


Fig. 3. (a) The rules never single out Merlin, yet human assassins beat a rules-only guess. 387 games that ended in an assassination (game counts in brackets). The dashed line is the success rate of picking uniformly among the players that the rules still allow to be Merlin. **(b) The counterexample-guided search scales; enumeration and adaptive search do not.** Mean time to find the smallest set of alignment checks that decides the full evil team, on 150 end-of-game states; enumeration and adaptive search were run up to seven players.

Most open questions take two or three checks (Figure 3b). On 150 end-of-game states where the public cannot yet name the evil team, the minimum number of alignment checks that decides the full team grows from 1.8 on average in five-player games to 2.9 in eight- and nine-player games and 4.0 in ten-player games. The counterexample-guided search finds it in 0.12–0.88 s on average. Enumeration returns the same sizes on all 108 states with seven or fewer players, but its cost grows with the number of subsets. Adaptive checking never needed fewer checks than the one-shot set (108 of 108 states equal) and costs 15 s per state at seven players. For deciding the evil team in these games, choosing the next check after seeing the last answer does not help.

7 Where Does a Language Model Help?

The course suggests using a language model to automate the workflow. The model could answer questions about the rules itself, or it could translate them into the formal model and let the solver answer. We compare these options, and two ways of combining them, on the same frozen questions.

7.1 Setup

Questions. We built 510 questions from 510 different real games (seed 137): 150 each whose correct verdict is *entailed*, *contradicted* or *undetermined*, and 60

inconsistent ones. For the inconsistent questions we corrupted one visible fact: a team one player too large (21), more Fail cards than team members (21), a Fail card played by the good player whose view is given (9), a two-Fail violation (5), too many evil players implied by the Fail counts (3), and Percival seeing three players (1). Each question is the record written out in Chinese (setup, every proposal with its approvers, every quest), a point of view (a public observer in 30% of questions, otherwise one player's true view), and a claim of one of eight types. The correct labels come from the verified Z3 model. For the 455 questions with at most eight players, we also re-derived each label with the brute-force oracle and found no disagreement.

Model and conditions. We use Qwen3-8B [16] at a fixed revision in BF16 on one GPU (an RTX PRO 6000 with 95 GiB). Every condition sees the same rule card, reproduced in Appendix B:

- **Direct**: answer with one word (greedy decoding).
- **Thinking**: reason first in thinking mode (temperature 0.6, top-p 0.95, top-k 20) within a budget of 3,072, 8,192 or 16,384 new tokens. A trace cut off at one budget was *continued from its own text* up to the next. Because sampling simply carries on from the same prefix, this gives exactly the distribution of a run with the larger budget, at a fraction of the cost. A trace that is still unfinished either counts as no answer (*strict*) or is closed and asked for its verdict (*forced*, as in budget forcing [13]).
- **Translate, then Z3 decides**: the model only converts the text into a JSON instance (players, roles, quests, point of view, claim), guided by one worked example, and the verifier classifies it.
- **Verify, then revise**: the model's direct answer is compared with Z3's verdict on the model's *own* translation. On disagreement the model is asked again, either told only that its answer is not supported (*flag*) or shown the solver's evidence: two worlds, or the clause core (*evidence*). No correct label is used anywhere.
- **Gold-selected revise** (an upper bound): only answers known to be wrong are asked again, with evidence from the correct formalisation.
- **Translation repair**: when Z3 finds the model's translation inconsistent, the conflicting clauses are fed back once and the model may correct its JSON, in the style of Logic-LM [15].

We report accuracy with 95% bootstrap intervals over questions (2,000 resamples [6]) and paired bootstrap intervals for differences. Table 4 gives every condition; the figures show the main ones.

7.2 Translate, don't reason

Answering directly, the model is right on half the questions (Figure 4). Its errors have a clear shape. It never says a record is illegal (0 of 60), and it calls 115 of the 150 contradicted claims undetermined. Thinking helps, to 67.1% with a 16k budget, mostly on contradicted claims, but it still catches only 11 of the

Table 4. All conditions on the 510 questions. *Premature*: open questions answered as settled (of 150). *False und.*: settled questions answered as open (of 300). *Missed inc.*: illegal records not flagged (of 60). *False inc.*: legal records flagged as illegal (of 450). *Unparsed*: answers that are not one of the four words.

Condition	Acc. %	Macro-F1	Premature	False und.	Missed inc.	False inc.	Unparsed
The model alone							
Direct	50.0	.370	28	152	60	0	0
Thinking 3,072, strict	47.6	.504	10	25	57	1	207
Thinking 3,072, forced	67.1	.617	26	80	47	1	5
Thinking 8,192, strict	66.9	.612	25	71	49	1	15
Thinking 16,384	67.1	.612	25	71	49	1	13
The solver in the loop (no correct labels used)							
Translate, then Z3 decides	**92.0**	**.908**	2	2	0	25	0
with translation repair	92.0	.908	2	2	0	25	0
Verify, revise (flag)	65.3	.506	12	27	60	0	0
Verify, revise (evidence)	89.2	.880	2	13	3	25	0
on repaired translation	89.0	.877	2	13	4	25	0
Upper bounds (questions to re-ask chosen with correct labels)							
Gold-selected (flag)	70.0	.544	0	15	60	0	0
Gold-selected (evidence)	96.9	.968	0	10	4	0	0

60 illegal records. Translating and letting Z3 decide reaches 92.0%: 42.0 points above direct answering (paired 95% interval $[36.9, 47.1]$) and 24.9 points above thinking ($[20.2, 30.0]$). It also flags all 60 illegal records.

The reason is that translation is local while reasoning is global. To translate, the model only has to copy what each sentence says. It gets all 510 quest records exactly right and 455 of the 510 questions right in every field. To reason, it has to keep every rule and every quest in view at once, and that is where it fails.

7.3 Thinking longer removes truncation, not errors

With the first run's budget of 3,072 tokens, 206 of the 510 traces were cut off before they reached an answer, and strict accuracy (47.6%) was below direct answering. Continuing those traces finishes all but two by 8,192 tokens and all of them by 16,384, and strict accuracy rises to 67.1% (Figure 5a). But forcing the unfinished traces to answer at 3,072 tokens already gives 67.1%. On the 206 questions that needed more than 3,072 tokens, the forced answers and the finished ones are equally accurate (49.0% each) and agree on 135 questions. The questions that finished early were answered correctly 79.3% of the time. Accuracy falls steadily with the length of the finished trace, from 86.5% under 2k tokens to 22% above 6k (Figure 5b). Long traces are a sign that the model is lost, not that it is making progress. So the low accuracy of the first run was mostly truncation, but the ceiling of about two-thirds is the model's. Thirteen finished traces gave the verdict followed by an explanation, which the instructions forbade, and our pre-registered parser counts these as unparsed. Reading only their first line would give 68.8%, so the conclusion does not depend on this choice.

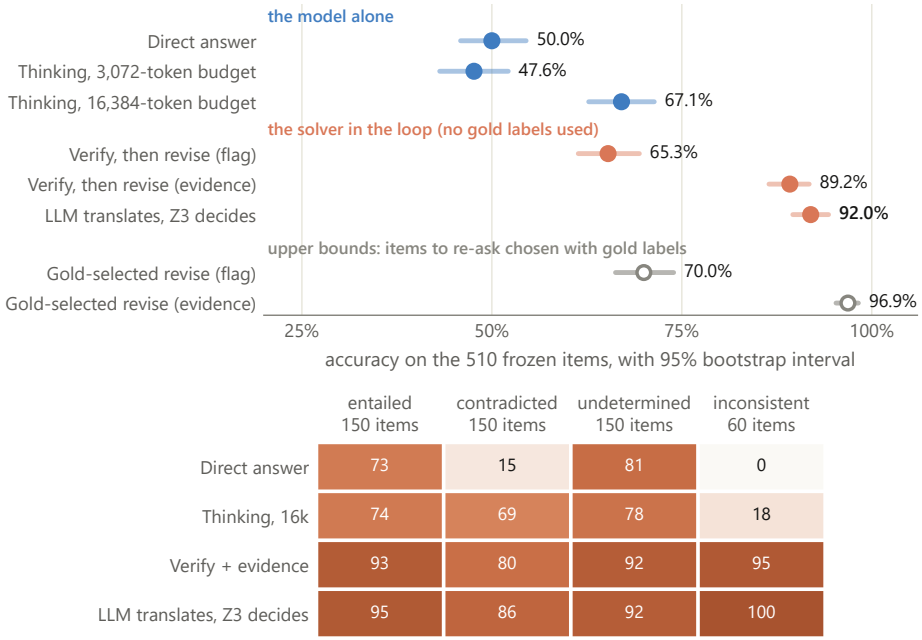


Fig. 4. The model is far more accurate as a translator than as a reasoner. Top: accuracy with 95% bootstrap intervals; open circles are upper bounds that need the correct labels. Bottom: share of questions of each correct verdict that a condition gets right (%). Answering directly, the model never recognises an illegal record and calls most contradicted claims open; with Z3 deciding, every illegal record is caught.

7.4 Translation errors are systematic, and the solver sees some of them

The translation errors are not random. All 27 errors in the role configuration occur in games of seven or more players that include Oberon and exactly one Minion: 27 of the 31 such questions, against 0 of the other 479. In each case the model adds a second Minion. It seems not to count Oberon as evil when it reconciles the role list with the stated number of evil players. The resulting configuration has one role card too many, so Z3 rejects every such translation as *inconsistent*. Those are the 25 false "illegal record" verdicts in Table 4; the other two fell on questions that were illegal anyway. The solver cannot see the other kind of error. In 27 questions the claim itself was mistranslated, for example "seat 1 must be Merlin" became "seat 1 is good", and the solver then answers a different question correctly.

Repair did not help. The 85 translations that Z3 rejected (60 genuinely illegal records and the 25 mistranslations) were sent back with the conflicting clause, for example "the role cards must add up to the number of players". In all 85 cases the model returned its JSON unchanged, byte for byte, including the 25 it

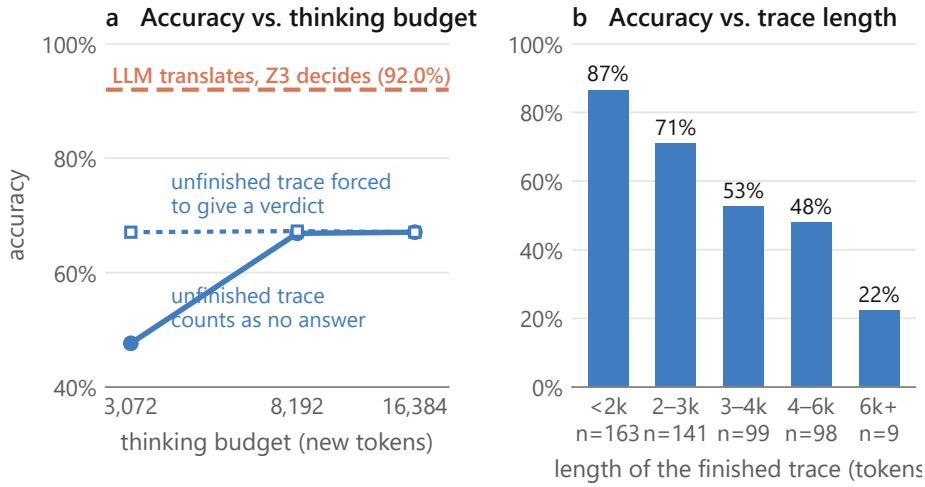


Fig. 5. More budget lets every trace finish but does not make the model more accurate. (a) Accuracy at three thinking budgets. At 3,072 tokens 206 traces are unfinished; at 8,192, two; at 16,384, none. Forcing an unfinished trace to answer already reaches the accuracy of letting it finish. (b) Accuracy of the finished traces by their length: long traces mark hard questions, not productive search.

had got wrong. The feedback was accurate and specific, but the misconception behind the error survived it. This matches the finding that language models rarely correct their own reasoning without new information [8]. Our prompt also allowed the model to return the same JSON "if the text itself breaks the rules", which gave it an easy way out.

7.5 The model defers to the solver, even when the solver is wrong

Comparing the direct answer with Z3's verdict on the model's own translation is an effective alarm. It flags 253 of the 255 wrong direct answers, at the cost of also flagging 23 of the 255 correct ones. What happens next depends on what the model is told (Figure 6). Told only that its answer is not supported, it changes its answer every time (0 of 276 kept) and lands on the solver's verdict 103 times, for 65.3% overall. Shown the solver's evidence, it adopts the solver's verdict 259 times: on 222 of the 237 questions where the solver was right, and on 37 of the 39 where the solver had been handed a mistranslated question. The result, 89.2%, is 2.7 points *below* simply outputting the solver's verdict (paired interval $[-4.3, -1.4]$). The 96.9% of the gold-selected variant is out of reach in practice, because it requires knowing which answers are wrong.

The evidence persuades whether or not it answers the right question, consistent with the broader tendency of models to defer to authoritative-sounding input [18]. The solver's evidence is valuable, but as something for a *person* to check, not as a prompt that makes the model's final answer more reliable.

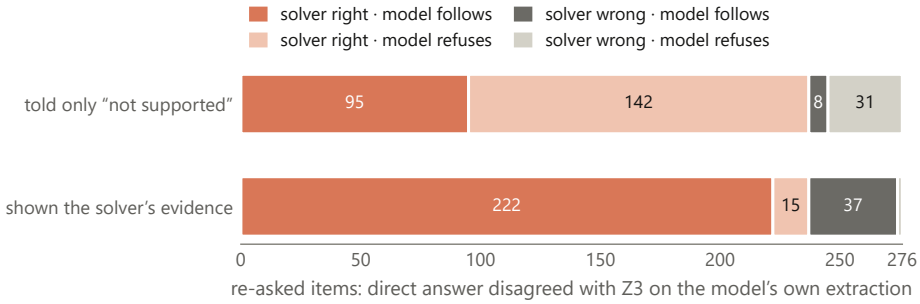


Fig. 6. Shown the solver's evidence, the model adopts the solver's verdict whether or not the solver was given the right question. The 276 questions asked again because the direct answer disagreed with Z3 on the model's own translation, split by whether the solver's verdict was correct and whether the model's new answer adopted it.

7.6 Proposing what to check

For the 122 open questions that have a sufficient reveal set, we also asked the model directly for the fewest players to check. Its proposals were sufficient 99 times (81.1%) and minimum 35 times (28.7%); two were invalid, and the sufficient ones used 0.96 more checks than necessary on average. That is useful as a candidate for the solver to verify with one TwoWorld query, not as an answer.

8 Limitations

- **Generated text.** The questions are records written out from templates. Real conversations are messier, so the translation accuracy here is an upper bound for free text.
- **Labels from our own model.** The correct labels come from our formalisation. It passes every check in Section 5, but the language-model study still measures agreement with our reading of the rulebook, including the defaults in Table 2.
- **One model, one sampling run.** The intervals reflect resampling of questions, not variation across seeds or models.
- **Which evidence is shown.** When several cores or witness pairs exist, the one Z3 returns depends on the solver's history in the process. We could reproduce all nine sets of follow-up prompts byte for byte by replaying the run's order of queries, but an independent re-implementation would show different, equally valid evidence.
- **Coverage.** The Lady of the Lake never occurs in the logs, so its four clauses are tested only by hand cases. Speech and voting strategy are outside the model by design.

9 Related Work

Checking conclusions against formalised rules. Amazon Bedrock's Automated Reasoning checks [1] formalise a policy document and classify a model's answer as valid, invalid or lacking information. Our verdicts refine this with a separate *inconsistent* case, checkable evidence and a minimum-information follow-up. Z3 [12] and the SMT-LIB language [3] provide the reasoning layer.

Language models with solvers. Logic-LM [15], SatLM [21] and LINC [14] translate natural language into solver input and outperform chain-of-thought on logical reasoning; Don't Trust: Verify [22] uses autoformalisation to check answers. Our results agree that translating is better than reasoning, and add three observations: solver feedback does not repair errors that come from a misconception, translation errors split into those the solver can see and those it cannot, and a model asked to judge the solver's verdict defers to it.

Explanations and hitting sets. Deletion-based minimal unsatisfiable subsets [10] and QuickXplain [9] explain over-constrained problems. Our minimum reveal search follows the implicit hitting-set approach [5] and the counterexample-guided pattern of CEGAR and CEGIS [4,19].

Avalon and hidden-role games. DeepRole [17] tracks beliefs over role assignments consistent with observations; our verifier is the exact, rules-only version of that consistency check. AvalonBench [11] and Stepputtis et al. [20] evaluate language models that play Avalon or identify roles from dialogue. Budget forcing [13] motivates our forced-answer condition.

10 Conclusion

The rules of Avalon prove less than players think, and a constraint solver can say exactly how much: which claims are forced, which are open, which records are impossible, why, and which checks would settle the rest. Getting there takes more than encoding sentences. The four verdicts have to be kept apart, ambiguous readings have to be made explicit, and the model has to be checked against an independent implementation and against real games. A language model asked to do this reasoning itself gets half of it right, or two-thirds after long thinking. Asked only to translate, with the solver doing the reasoning, it gets 92%. The errors that remain are translation errors. The solver exposes some of them, the model does not fix them when told, and it will defer to a solver that has been given the wrong question. For verification pipelines that use both, the practical rule is simple: let the model translate, let the solver decide, and show people the solver's evidence.

11 Answers to the Course Questions

Note to the author (delete before submission). The course requires this section to be typed by the author personally, without generative AI. It has deliberately been left empty. Question 5 must disclose every place where generative AI was used in this project.

1. 如果你现在正在某个研究生岗位/工作岗位的面试阶段，当面试者问起你的这个项目时，尝试用一段话介绍你的项目。

2. 从你完成项目的过程来看，你认为约束求解器在各种场景下能够证明什么，不能证明什么？

3. 为什么约束求解可以（或者不完全可以）用于验证一个结论是否符合背景知识？

4. 将自然语言规则建模为形式约束时，可能会出现哪些问题？

5. 披露你的项目在哪些环节（如报告撰写，程序编写，工作流当中等）使用了生成式人工智能，说明你是如何使用的。

References

1. Amazon Web Services: Automated reasoning checks in Amazon Bedrock guardrails. `https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-automated-reasoning-checks.html`, accessed November 2026
2. avalonlogs: logs of online games of the resistance: Avalon. GitHub repository `WhoaWhoa/avalonlogs`, commit `3219bbd`, accessed November 2026
3. Barrett, C., Fontaine, P., Tinelli, C.: The SMT-LIB standard: Version 2.6. Tech. rep., Department of Computer Science, The University of Iowa (2017), available at `https://smt-lib.org`
4. Clarke, E., Grumberg, O., Jha, S., Lu, Y., Veith, H.: Counterexample-guided abstraction refinement. In: Computer Aided Verification (CAV). LNCS, vol. 1855, pp. 154–169. Springer (2000)
5. Davies, J., Bacchus, F.: Solving MAXSAT by solving a sequence of simpler SAT instances. In: Principles and Practice of Constraint Programming (CP). LNCS, vol. 6876, pp. 225–239. Springer (2011)
6. Efron, B., Tibshirani, R.J.: An Introduction to the Bootstrap. Chapman & Hall (1993)
7. Eskridge, D.: The resistance: Avalon. rulebook. Indie Boards & Cards (2012)
8. Huang, J., Chen, X., Mishra, S., Zheng, H.S., Yu, A.W., Song, X., Zhou, D.: Large language models cannot self-correct reasoning yet. In: International Conference on Learning Representations (ICLR) (2024)
9. Junker, U.: QuickXplain: Preferred explanations and relaxations for over-constrained problems. In: Proceedings of the 19th National Conference on Artificial Intelligence (AAAI). pp. 167–172 (2004)
10. Liffiton, M.H., Sakallah, K.A.: Algorithms for computing minimal unsatisfiable subsets of constraints. Journal of Automated Reasoning **40**(1), 1–33 (2008)
11. Light, J., Cai, M., Shen, S., Hu, Z.: AvalonBench: Evaluating LLMs playing the game of Avalon. arXiv preprint arXiv:2310.05036 (2023)
12. de Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). LNCS, vol. 4963, pp. 337–340. Springer (2008)
13. Muennighoff, N., Yang, Z., Shi, W., Li, X.L., Fei-Fei, L., Hajishirzi, H., Zettlemoyer, L., Liang, P., Candès, E., Hashimoto, T.: s1: Simple test-time scaling. arXiv preprint arXiv:2501.19393 (2025)
14. Olausson, T.X., Gu, A., Lipkin, B., Zhang, C.E., Solar-Lezama, A., Tenenbaum, J.B., Levy, R.: LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP) (2023)
15. Pan, L., Albalak, A., Wang, X., Wang, W.Y.: Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. In: Findings of the Association for Computational Linguistics: EMNLP 2023 (2023)
16. Qwen Team: Qwen3 technical report. arXiv preprint arXiv:2505.09388 (2025)
17. Serrino, J., Kleiman-Weiner, M., Parkes, D.C., Tenenbaum, J.B.: Finding friend and foe in multi-agent games. In: Advances in Neural Information Processing Systems (NeurIPS) (2019)
18. Sharma, M., Tong, M., Korbak, T., Duvenaud, D., Askell, A., Bowman, S.R., et al.: Towards understanding sycophancy in language models. In: International Conference on Learning Representations (ICLR) (2024)

19. Solar-Lezama, A., Tancau, L., Bodík, R., Seshia, S., Saraswat, V.: Combinatorial sketching for finite programs. In: Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). pp. 404–415 (2006)
20. Stepputtis, S., Campbell, J., Xie, Y., Qi, Z., Zhang, W.S., Wang, R., Rangreji, S., Lewis, C., Sycara, K.: Long-horizon dialogue understanding for role identification in the game of Avalon with large language models. In: Findings of the Association for Computational Linguistics: EMNLP 2023 (2023)
21. Ye, X., Chen, Q., Dillig, I., Durrett, G.: SatLM: Satisfiability-aided language models using declarative prompting. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)
22. Zhou, J.P., Staats, C., Li, W., Szegedy, C., Weinberger, K.Q., Wu, Y.: Don't trust: Verify – grounding LLM quantitative reasoning with autoformalization. In: International Conference on Learning Representations (ICLR) (2024)

A All Hand-Written Test Cases

The table below is generated from the test file `avalon/cases.py` by the script `report/tables/make_tables.py`. The script also re-runs each case through Z3 and stops if any verdict differs from the expected one.

Table 5: All 44 hand-written test scenarios, quoted as written (in Chinese) in `avalon/cases.py`. *Claim* is the formalised conclusion (E: the set of evil seats; r_i: the role of seat i; seats are 1-based). *Exp.* is the verdict derived by hand from the rulebook; Z3 and the independent oracle return exactly this verdict for every row.

ID	Kind	Scenario and question (verbatim)	Claim	Clauses	Exp.
T01	simple	7 人局，角色为梅林、派西维尔、忠臣×2、莫甘娜、莫德雷德、爪牙。问：本局有几名坏人？答：3 名。	$\lvert\{1,2,3,4,5,6,7\} \cap E\rvert = 3$	`S-COUNT`, `S-DEAL`	ent.
T02	simple	同上。答：2 名坏人。	$\lvert\{1,2,3,4,5,6,7\} \cap E\rvert = 2$	`S-COUNT`, `S-DEAL`	con.
T03	illegal	6 人局却配置了莫甘娜、莫德雷德、爪牙三名坏人。问：1 号是好人吗？答：是。	$\neg evil_1$	`S-COUNT`	inc.
T04	simple	8 人局第 1 个任务需要几人出队？答：3 人。	team $\text{size}_{q1} = 3$	`T-SIZE`	ent.
T05	boundary	5 人局第 3 个任务需要几人出队？答：3 人。	team $\text{size}_{q3} = 3$	`T-SIZE`	con.
T06	illegal	5 人局第 1 个任务由 1、2、3 号三人执行，0 张失败票。问：4 号是好人吗？答：是。	$\neg evil_4$	`Q-SIZE`	inc.
T07	boundary	7 人局第 4 个任务翻开 1 张失败票。问：任务失败了吗？答：失败。	quest 4 fails with 1 Fail	`Q-TWOFAIL`	con.
T08	boundary	6 人局第 4 个任务翻开 1 张失败票。问：任务失败了吗？答：失败。	quest 4 fails with 1 Fail	`Q-TWOFAIL`	ent.
T09	boundary	7 人局第 4 个任务翻开 2 张失败票。问：任务失败了吗？答：失败。	quest 4 fails with 2 Fail	`Q-TWOFAIL`	ent.
T10	boundary	7 人局第 3 个任务翻开 1 张失败票。问：任务失败了吗？答：失败。	quest 3 fails with 1 Fail	`Q-TWOFAIL`	ent.
T11	illegal	7 人局记录：前三个任务正常，第 4 个任务 4 人出队、1 张失败票，却被记为失败。问：1 号是坏人吗？答：是。	$evil_1$	`Q-RESULT`, `Q-TWOFAIL`	inc.
T12	boundary	6 人局，提案得到 3 票赞成。问：提案通过了吗？答：通过。	3 approvals pass	`T-VOTE`	con.
T13	boundary	7 人局，提案得到 4 票赞成。问：提案通过了吗？答：通过。	4 approvals pass	`T-VOTE`	ent.
T14	illegal	8 人局第 1 轮：队长 1 号提名 1、2、3 号，赞成者为 1、2、3、4 号，记为通过。问：5 号是好人吗？答：是。	$\neg evil_5$	`T-VOTE`	inc.
T15	illegal	5 人局第 1 轮连续 5 次提案都被否决，但随后第 2 轮仍继续进行。问：1 号是好人吗？答：是。	$\neg evil_1$	`T-HAMMER`	inc.
T16	illegal	6 人局第 1 轮：1 号提案被否决后，下一次提案的队长是 3 号。问：1 号是好人吗？答：是。	$\neg evil_1$	`T-LEADER`	inc.
T17	simple	5 人局第 1 个任务由 1、2 号执行，翻开 2 张失败票。问：1 号和 2 号都是坏人吗？答：是。	$evil_1 \wedge evil_2$	`Q-GOOD`, `Q-COUNT`	ent.
T18	combin.	同上。问：3 号是好人吗？答：是（坏人只有 2 名，已全部在队中）。	$\neg evil_3$	`Q-GOOD`, `Q-COUNT`, `S-COUNT`	ent.
T19	simple	5 人局第 1 个任务由 1、2 号执行，翻开 1 张失败票。问：队里恰好有一个坏人吗？答：是。	$\lvert\{1,2\} \cap E\rvert = 1$	`Q-GOOD`, `Q-COUNT`	und.
T20	simple	同上。问：队里至少有一个坏人吗？答：是。	$\lvert\{1,2\} \cap E\rvert \geq 1$	`Q-GOOD`, `Q-COUNT`	ent.

ID	Kind	Scenario and question (verbatim)	Claim	Clauses	Exp.
T21	simple	5 人局第 1 个任务由 1、2 号执行，0 张失败票，任务成功。问：1 号和 2 号都是好人吗？答：是。	$\neg evil_1 \wedge \neg evil_2$	`Q-GOOD`	und.
T22	combin.	7 人局第 2 个任务由 1、2、3 号执行，翻开 3 张失败票。问：4 到 7 号都是好人吗？答：是。	$\neg evil_i$ for $i \in \{4,\ldots,7\}$	`Q-COUNT`, `S-COUNT`	ent.
T23	combin.	6 人局：第 1 个任务（1、2 号）1 张失败票；第 2 个任务（3、4、5 号）1 张失败票。问：6 号是好人吗？答：是。	$\neg evil_6$	`Q-COUNT`, `S-COUNT`	ent.
T24	combin.	同上。问：1 号是坏人吗？答：是。	$evil_1$	`Q-COUNT`, `S-COUNT`	und.
T25	illegal	6 人局：第 1 个任务（1、2 号）2 张失败票；第 2 个任务（3、4、5 号）1 张失败票。问：6 号是好人吗？答：是。	$\neg evil_6$	`Q-COUNT`, `S-COUNT`	inc.
T26	view	7 人局（含莫德雷德）。你是 1 号梅林，开局看到 3 号、5 号是坏人。问：坏人就是 3 号和 5 号吗？答：是。	$E = \{3, 5\}$	`K-MERLIN`, `S-DEAL`	con.
T27	view	同上。问：2、4、6、7 号中恰好还有一名坏人吗？答：是（莫德雷德对梅林隐藏）。	$\lvert\{2, 4, 6, 7\} \cap E\rvert = 1$	`K-MERLIN`, `S-COUNT`	ent.
T28	combin.	同上，另外第 1 个任务由 2、4 号执行，翻开 1 张失败票。问：6 号和 7 号都是好人吗？答：是。	$\neg evil_6 \wedge \neg evil_7$	`K-MERLIN`, `Q-COUNT`, `S-COUNT`	ent.
T29	view	7 人局。你是 2 号派西维尔，开局看到 4 号和 6 号（梅林与莫甘娜）。问：4 号是梅林吗？答：是。	$r_4 = \textsc{Merlin}$	`K-PERCIVAL`	und.
T30	combin.	同上，另外第 1 个任务由 4、7 号执行，翻开 2 张失败票。问：6 号是梅林吗？答：是。	$r_6 = \textsc{Merlin}$	`K-PERCIVAL`, `Q-COUNT`	ent.
T31	view	7 人局（含奥伯伦）。你是 3 号爪牙，开局只看到 5 号一名同伴。问：坏人只有你和 5 号吗？答：是。	$E = \{3, 5\}$	`K-EVIL`, `S-COUNT`	con.
T32	view	同上。问：1 号是好人吗？答：是。	$\neg evil_1$	`K-EVIL`	und.
T33	combin.	5 人局。你是 1 号忠臣。第 1 个任务由 2、3 号执行，翻开 2 张失败票。问：4 号和 5 号都是好人吗？答：是。	$\neg evil_4 \wedge \neg evil_5$	`V-SELF`, `Q-COUNT`, `S-COUNT`	ent.
T34	illegal	7 人局。你是 2 号派西维尔，开局看到了 3 号、4 号、6 号三个人。问：3 号是梅林吗？答：是。	$r_3 = \textsc{Merlin}$	`K-PERCIVAL`	inc.
T35	illegal	5 人局。你是 1 号忠臣，和 2 号一起执行第 1 个任务，翻开 2 张失败票。问：2 号是坏人吗？答：是。	$evil_2$	`V-SELF`, `Q-GOOD`, `Q-COUNT`	inc.
T36	simple	5 人局，前三个任务成功；刺客刺杀 4 号后好人获胜。问：4 号不是梅林吗？答：不是梅林。	$\neg(r_4 = \textsc{Merlin})$	`G-MERLIN-HIT`	ent.
T37	simple	同样的任务结果，但刺杀 4 号后坏人获胜。问：4 号是梅林吗？答：是。	$r_4 = \textsc{Merlin}$	`G-MERLIN-HIT`	ent.
T38	illegal	5 人局只成功了两个任务，刺客就开始刺杀。问：4 号是梅林吗？答：否。	$\neg(r_4 = \textsc{Merlin})$	`G-ASSASSIN`	inc.
T39	illegal	5 人局前三个任务都失败后，记录里还有第 4 个任务。问：1 号是坏人吗？答：是。	$evil_1$	`G-END`	inc.
T40	view	8 人局。你是 1 号忠臣，用湖中仙女查验 4 号，结果为好人。问：4 号是好人吗？答：是。	$\neg evil_4$	`K-LADY`	ent.
T41	illegal	8 人局首位队长 1 号，湖中仙女在 8 号手中。第 2 轮后 8 号查验 3 号；第 3 轮后 3 号又查验 8 号。问：3 号是好人吗？答：是。	$\neg evil_3$	`L-PREV`, `L-CHAIN`	inc.
T42	variant	（只公开任务结果、不公开失败票数的读法）5 人局第 1 个任务由 1、2 号执行并失败。问：1 号和 2 号都是坏人吗？答：是。	$evil_1 \wedge evil_2$	`Q-COUNT`, `V-FAILCOUNT`	und.

ID	Kind	Scenario and question (verbatim)	Claim	Clauses	Exp.
T43	variant	10 人局（含莫德雷德、奥伯伦）。你是 1 号梅林，开局看到 3、5、7 号三名坏人。问：2、4、6、8、9、10 号中恰好还有一名坏人吗？答：是。	$\lvert\{2,4,6,8,9,10\} \cap E\rvert = 1$	`K-MERLIN`, `V-OBERON`	ent.
T44	variant	同上，但按“奥伯伦不向梅林显示”的读法。答：是。	$\lvert\{2,4,6,8,9,10\} \cap E\rvert = 1$	`K-MERLIN`, `V-OBERON`	inc.

B Prompts

Generated verbatim from `avalon/llm_run.py` and `avalon/followup.py`. The model saw these texts in Chinese, the language of the records.

System prompt for direct answers and thinking (rule card + task)

```
你在分析一局《阿瓦隆》（The Resistance: Avalon）。只能依据下列规则和给出的记录做严格的逻辑推理；
↪ 发言、投票倾向、"像不像"之类的策略性推测都不算证据。

【阵营与人数】好人阵营：梅林、派西维尔、亚瑟的忠臣。坏人阵营：莫甘娜、莫德雷德、奥伯伦、
↪ 莫德雷德的爪牙。坏人人数：5人局2名，6人局2名，7人局3名，8人局3名，9人局3名，10人局4名。
↪ 本局使用的角色在记录开头给出，每人一张。
【开局信息】每个人知道自己的角色。除奥伯伦外的坏人互相认识，他们看不到奥伯伦，奥伯伦也看不到他们。
↪ 梅林能看到除莫德雷德以外的所有坏人（包括奥伯伦）。派西维尔能看到梅林和莫甘娜，但分不清谁是谁。
↪ 亚瑟的忠臣和奥伯伦开局得不到任何信息。
【组队与投票】队长提名队伍，所有人公开投票，赞成票严格过半才通过（平票算否决）。
↪ 每次提案后队长按座位顺序交给下一位。同一轮连续5次否决则坏人直接获胜。投票可以随意投，
↪ 规则上与身份无关。各轮出队人数：5人局2,3,2,3,3；6人局2,3,4,3,4；7人局2,3,3,4,4；
↪ 8至10人局3,4,4,5,5。
【执行任务】每名队员出一张任务牌：好人必须出成功；坏人可以出成功，也可以出失败。牌洗混后翻开，
↪ 所有人知道失败票的张数，但不知道是谁出的。出现1张失败票任务就失败；例外：
↪ 7人及以上的第4个任务需要至少2张失败票才失败。
【结束】三个任务失败则坏人获胜。三个任务成功后刺客刺杀一人：刺中梅林则坏人获胜，否则好人获胜。
【湖中仙女】持有者查验一名玩家，得知其阵营（好人或坏人），结果只有持有者本人知道。

判断给定" 说法" 相对于规则和记录（以及你的视角信息）的地位，只能回答一个词：
成立：在所有符合规则与记录的身份分配中，说法都为真；
不成立：在所有这样的身份分配中，说法都为假；
不确定：有的身份分配下为真，有的为假；
矛盾：记录或你的视角信息本身违反规则，不存在任何符合的身份分配。
```

Answer instruction appended to every question

```
只输出 成立、不成立、不确定、矛盾 中的一个词，不要解释。
```

Extraction system prompt (followed by one worked example)

```
把一段《阿瓦隆》局面描述逐字转换成 JSON，不做任何推理。格式：
{"n": 人数,
 "config": {角色英文名: 数量, ...}（只列出现的角色）,
 "quests": [{"q": 任务编号, "team": [座位号], "fails": 失败票张数, "result":
 ↪ "success"或"fail"}]（只列已执行的任务；提案和投票不需要）,
 "view": null 或 {"player": 座位号, "role": 角色英文名, "seen": [开局看到的座位号], "lady":
 ↪ [[被查验的座位号, 是坏人则true否则false]]},
 "claim": 说法的 JSON}
```

```
说法的写法：X号一定是坏人 → {"t":"evil","p":X}；X号一定是好人 → {"t":"good","p":X}；
↪  X号一定是梅林 → {"t":"role","p":X,"r":"merlin"}；X号一定不是梅林 →
↪ {"t":"not","a":{"t":"role","p":X,"r":"merlin"}}；某些人中至少有k名坏人 →
↪ {"t":"atleast","ps":[座位号],"k":k}；某些人中恰好有k名坏人 →
↪ {"t":"exactly","ps":[座位号],"k":k}；X号和Y号一定属于同一阵营 → {"t":"same","ps":[X,Y]}；
↪  本局的坏人恰好是某些人 → {"t":"evil_set","ps":[座位号]}。
角色英文名：梅林 merlin，派西维尔 percival，亚瑟的忠臣 servant，莫甘娜 morgana，莫德雷德
↪  mordred，奥伯伦 oberon，莫德雷德的爪牙 minion。座位号使用文字中的号码（从1开始）。
↪  旁观者视角时 view 为 null。只输出一个JSON对象，不要解释，不要Markdown。
```

The worked extraction example: input

```
6 人局，角色为：梅林、派西维尔、亚瑟的忠臣 ×2、莫甘娜、莫德雷德的爪牙（共 2 名坏人）。
第 1 轮第 1 次提案：队长 3 号提名 1 号、6 号，赞成者为 1 号、3 号、6 号，被否决。
第 1 轮第 2 次提案：队长 4 号提名 2 号、4 号，赞成者为 1 号、2 号、4 号、5 号，通过。
第 1 个任务由 2 号、4 号执行，翻开 1 张失败票，任务失败。
你是 5 号，身份是派西维尔。开局时你看到 1 号、3 号（其中一人是梅林，另一人是莫甘娜，你分不清）。
说法：2 号和 4 号中至少有 1 名坏人。
```

The worked extraction example: output

```
{"n": 6, "config": {"merlin": 1, "percival": 1, "servant": 2, "morgana": 1, "minion": 1},
↪  "quests": [{"q": 1, "team": [2, 4], "fails": 1, "result": "fail"}], "view": {"player":
↪  5, "role": "percival", "seen": [1, 3], "lady": []}, "claim": {"t": "atleast", "ps":
↪  [2, 4], "k": 1}}
```

Reveal-set question

```
现在可以用湖中仙女查验若干玩家的阵营（每查验一人，就得知他是好人还是坏人）。
↪  请一次性给出最少需要查验的玩家，使得无论查验结果如何，都一定能判断上述说法成立还是不成立。
↪  不能查验你自己。只输出座位号的JSON数组，例如 [3, 5]，不要解释。
```

Budget-forcing suffix (appended to an unfinished thinking trace)

```
考虑到思考时间有限，我现在必须根据已有的思考直接给出最终答案。
</think>
```

Extraction-repair instruction (after the solver-reported conflict)

```
请逐项对照原文检查你的转写：人数、角色配置、每个任务的队员、失败票张数和结果、你的视角信息、说法。
↪  如果转写有误，输出改正后的完整JSON；如果转写与原文一致（也就是原文记录本身违反规则），
↪  就原样输出同一个JSON。只输出一个JSON对象，不要解释，不要Markdown。
```

C Implementation and Reproducibility

Commands.

- `python avalon/selfcheck.py`: all checks of Table 3 (2,236 s on a laptop).
- `python avalon/followup.py --score`: re-scores the language-model runs from the stored outputs.
- `python avalon/followup.py --reconstruct`: rebuilds every follow-up prompt and checks it against the recorded hash.
- `python report/figures/make_figures.py`: every chart and the statistics quoted in the text.

Table 6. Code map. Everything runs on CPU except the language-model conditions.

File	Role
`avalon/smt.py`	Z3 encoding with clause-tagged constraints; four verdicts; witnesses; deletion-minimised cores; two-world test
`avalon/oracle.py`	independent brute-force reading of the rules (no shared rule code)
`avalon/search.py`	minimum reveal sets: enumeration, CEGAR with implicit hitting sets, adaptive minimax
`avalon/cases.py`	the 44 hand-written scenarios with formalisations and expected verdicts
`avalon/selfcheck.py`	all checks of Table 3
`avalon/sim.py`, `logs.py`, `analyze.py`	simulator, log parser and audit, real-game analyses (Section 6)
`avalon/llm_tasks.py`, `llm_run.py`, `followup.py`	frozen question set, prompts, model runner, scoring, follow-up conditions
`report/figures/*.py`, `report/tables/*.py`	every figure and generated table of this report

Fixed inputs. Z3 4.16.0.0; `avalonlogs` commit `3219bbd`; question set SHA-256 `256fefe3…c59c0`; Qwen3-8B revision `b968826`; seeds 137 (questions), 42 (sampling), 271 (analysis and bootstrap), 20261112 (self-check). The full run reports, including every interruption and how it was handled, are `RUN_REPORT.md` and `FOLLOWUP_RUN_REPORT.md`.